Deliverable — BTI note (T7, reference only)

Marius flagged BTI as background, not a workstream, so this is deliberately short.

Branch Target Identification (BTI) is ARM's coarse control-flow-integrity feature, introduced in Armv8.5-A (same architecture version as MTE). It's ARM's analogue to Intel CET's indirect-branch tracking (IBT).

- **What it does:** indirect branches (calls/jumps through a register) may only land on a special `BTI` landing-pad instruction. Jump anywhere else and the CPU faults. This stops redirecting an indirect branch into the middle of a function or a ROP/JOP gadget that doesn't start with a landing pad.
- **How it relates to the rest:** BTI is forward-edge CFI (where indirect calls/jumps go). PAC's return-address signing is backward-edge CFI (where returns go). Complementary, intended to be used together. Neither touches data; both sit at the "Hijack" stage, not the "Corrupt" stage where MTE lives.
- **Coarse, not fine:** BTI only checks the target is *a* valid landing pad, not the *right* one. It narrows the gadget space rather than eliminating it (much like PAC's default C-function-pointer schema allows substitution). A large binary has many landing pads.
- **Why it's only reference for us:** our targets are heap-memory-safety bugs under MTE and Scudo. BTI matters only if an exploit reaches control-flow hijack. For an OOB-read disclosure bug like the dav1d candidate, BTI is irrelevant. It becomes relevant only if a later target involves a write primitive aiming at a function pointer. Keep it in the back pocket.

To verify when it matters: whether Android ships user-space binaries built with BTI by default, and on which devices. Not confirmed — a build-flag question like the PAC-coverage one.